

Catch, avoid and shoot

Overview

The first action-game chapter. Students meet one genre three times - a fruit catcher, a space shooter, and a space shooter with levels, bosses and power-ups - and see how each version grows from the last. The Phaser ideas are **spawn timers** (`this.time.addEvent`), **object pools** (a physics group with `classType` and `maxSize` , `get()` , `enableBody` / `disableBody`), **waves** built from tweens, timers and a curved path, a **fire rate**, **invulnerability**, particles, a `TileSprite` starfield, and (in the advanced version) levels as data, a boss, power-ups and a HUD scene fed by events. The design ideas - readable threats, rising pressure, fairness - matter as much as the code.

The one idea to make sure every student leaves with is **pooling**, including its price: an object that is reused must be *fully* reset every time. Most of the chapter's bugs, and most of the students' bugs, come from that.

Prerequisites

- Chapter 2: scene keys, `init()` and the restart trap, `scene.start(key, data)`
- Chapters 4 and 6: launching a scene in parallel; events between scenes; `localStorage`
- Chapter 5: the game-wide sound manager (for the advanced version's music)
- Chapter 7: tweens, animations, particles
- Chapter 8: Arcade bodies, groups, `overlap` , process callbacks, `as` casts in callbacks
- Chapter 12 or 13: a small state machine held in a union type

Learning outcomes

Students can:

1. keep a player on one line with velocity and `setCollideWorldBounds` , driven by held keys

2. spawn objects on a timer with `this.time.addEvent({ delay, loop | repeat, callback })` , and stop it
3. explain why a shooter pools its bullets, and build a pool with a physics group, `get()` , `enableBody(true, x, y, true, true)` and `disableBody(true, true)`
4. explain why `killAndHide` is not enough for a physics object, and why a pooled object must be fully reset
5. build waves from staggered tweens, repeating timers and a tween along a path, and count a wave down to its end
6. add a fire rate, flashing invulnerability with a process callback, explosions with particles, and a difficulty ramp
7. describe how a game grows: data-driven levels, a phase state machine, one job per class, a HUD that only listens

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Play all three versions on the projector, back to back. Ask: "what stayed the same?"
0:05 - 0:15	Slides 3-6: the genre, the three versions, the catcher - player, timer, falling groups
0:15 - 0:35	Slides 7-11: pooling. Live demo of <code>killAndHide</code> (below). The three switches table
0:35 - 0:45	Slides 12-14: waves and counting a wave down, invulnerability, the starfield
0:45 - 0:55	Slides 15-18: the advanced version - data, phases, the swoop path, the boss, power-ups, HUD
0:55 - 1:00	Slides 19-20: summary, the challenges, and who should start where

Lab (2 hours)

Time	Activity
0:00 - 0:15	Everyone builds and plays the catcher; change <code>SPAWN_DELAY</code> , <code>BOMB_CHANCE</code> and <code>FALL_GRAVITY</code> and feel the difference
0:15 - 0:50	Challenges 1 and 2 (catcher)
0:50 - 1:00	Group check-in: open the intermediate version and find all three places a pooled object is switched on or off
1:00 - 2:00	Challenge 3 or 6 (intermediate), then 4 or 5 (advanced) for those who get there

Key points to stress

- **Pool what you make many of, often.** Bullets and enemies are pooled; explosions and power-ups are not. Ask students to justify each - it is a judgement, not a rule
- **Three switches:** `active` (updates, and "free" to the pool), `visible` (drawn), `body.enable` (moves and collides). `disableBody(true, true)` turns off all three; `killAndHide` only two
- **Reset everything on reuse.** Tweens (`killTweensOf`), velocities (`enableBody` with `reset` clears them), angles, tints, fields such as a bullet's owner
- **Count the wave; don't ask the pool.** `countActive() === 0` is true in the gaps of a stream wave
- `get()` **can return** `null` - type it as `Bullet | null` so the compiler insists on a check
- **The player's class does not know its keys.** The scene reads input and calls `move(-1 | 0 | 1)`. Challenges 1 and 6 both depend on this, and neither changes the ship or basket class
- **Fairness is design, not luck:** small hit boxes (`setBodySize`), invulnerability after a hit, enemies that stop firing when close

Common problems and errors

What students see	Cause
<code>TypeError: Cannot read properties of null (reading 'fire')</code> in the console, and the game freezes, when firing fast	<code>this.bullets.get()</code> as <code>Bullet</code> with no null check: the pool is full
Invisible bullets still destroy enemies	switched off with <code>killAndHide</code> or <code>setVisible(false)</code> - the body still on
A reused enemy flies off to an old position, or starts moving at the wrong speed	a tween from its last life is still running
The next wave never comes, or two come at once	<code>enemiesLeft</code> not decremented on every exit (shot, crash, off screen, pool empty), or decremented twice
The basket falls off the bottom of the screen	gravity set in the game config rather than on the falling groups
<code>TS2345 [ERROR]: Argument of type '(bullet: Bullet, enemy: Enemy) => void' is not assignable to parameter of type 'ArcadePhysicsCallback'. followed by Type 'Body StaticBody GameObjectWithBody Tile' is not assignable to type 'Bullet'.</code>	typing the overlap callback's parameters as your classes
<code>TS4114 [ERROR]: This member must have an 'override' modifier because it overrides a member in the base class 'TileSprite'.</code>	<code>preUpdate</code> in a <code>TileSprite</code> subclass without <code>override</code>

What students see	Cause
TS2741 [ERROR]: Property 'shield' is missing in type '{ spread: string; rapid: string; life: string; }' but required in type 'Record<PowerUpKind, string>'. (challenge 4)	a new kind added to the union but not to the <code>Record</code>
Things can still be caught after GAME OVER	spawn timer not removed, physics not paused
The second game starts with the old score, or ends at once	fields reset only where declared
The HUD works, but after a few games the game slows, or a HUD value updates several times	HUD listeners on the game scene's emitter never removed. There is error message - in the console, <code>game.scene.getScene("GameScene").events.listenerCount("s changed")</code> shows 2 after the second game, 3 after the third...
Holding SPACE fires one shot, a pause, then a fast stream (the keyboard's auto-repeat)	<code>keyboard.on("keydown-SPACE")</code> used for firing

The first error and the three `TS` errors above were produced by making those mistakes in copies of the projects; the listener count was read the same way, by removing the `SHUTDOWN` handler.

Suggested demos and live-coding moments

- `killAndHide` **is not enough**. In the intermediate version, change the body of `Bullet.kill()` from `this.disableBody(true, true);` to `this.setActive(false).setVisible(false);` - which is what `killAndHide` does. Play: bullets still vanish when they hit, and it looks fine - until a stream wave lines ships up in a column, and one shot destroys the whole column. Ask why before explaining. We checked this: one bullet fired up a column of three ships destroyed all three (score 300), with no bullet visible after the first.

- **The leftover tween.** Remove `this.scene tweens.killTweensOf(this);` from `Enemy.kill()` , shoot ships while they are still swooping into the row, and watch the next stream wave: some ships fly up into a row that is not there. We checked this: an enemy switched off mid-swoop and handed out again at (700, 500) was dragged straight back up to the row.
 - **Pool size.** Set `BULLET_POOL_SIZE` to 2 in the intermediate version: firing becomes a staccato pair. Then remove the `null` check - the game stops with the `TypeError` .
 - **Tuning live.** Open `levels.ts` in the advanced version, make level 1's boss 5 health and the first wave 20 ships. "Level design without touching game code."
-

Discussion questions

- Explosions are not pooled. Should they be? What would you need to reset? (Animation, position, and the `ANIMATION_COMPLETE` listener - a pooled sprite would need `once` re-added each time, or a single `on` added when made.)
 - Why does the intermediate version count `enemiesLeft` instead of asking `enemies.countActive() === 0` ? When would `countActive` be fine?
 - Power-ups are "active until" times on the clock, not timers. What are the advantages? What would go wrong with a `delayedCall` that switched spread shot off, if the player picked up a second one?
 - `WavePattern` is `"row" | "stream" | "swoop"` . In Java you would use an `enum` . What does each give you that the other does not?
 - The HUD only listens to events. What would be lost if it read `GameScene` 's fields every frame instead?
 - Where is the line between "fair" and "easy"? Which of the fairness rules in the chapter would you remove for an "expert" mode?
-

Extension ideas

- enemy variety: a class hierarchy (`Enemy` -> `Diver` , `Tank`) with different health and scores, still in one pool per class
- a boss with phases: a new attack pattern below half health
- replace the row wave's tween with `Phaser.GameObjects.PathFollower` and compare

- a pause scene (Chapter 4) that also pauses the music
 - screen shake strength that scales with what exploded; a hit-stop (pause physics for 50 ms)
-

Assessment ideas

- Practical: "add a fourth wave pattern of your own design to the advanced version's `WaveSpawner` , and use it in `levels.ts` " - look for the new pattern in the union type, a reset-safe enemy, and correct wave counting
 - Code reading: show `Enemy.kill()` without `killTweensOf` and a description of the bug; ask for the cause and the fix
 - Short answer: "What does each of `setActive(false)` , `setVisible(false)` and `disableBody()` switch off? Which does a pooled bullet need, and why?"
 - Short answer: "Why is a fire rate a stored time rather than a `delayedCall` ?"
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment (and `<!-- CHALLENGE n -->` in `index.html`). All six were played with the test harness, as described under each one.

1. Pointer control

Project: `solutions/ch14_challenge_1_pointer_control` (from `ch14_catcher_simple`)

Goal: scene-wide pointer events, and moving an object *towards* a target rather than onto it.

The pointer sets a target; the keys, when pressed, take over again:

`src/scenes/GameScene.ts`

```
this.input.on("pointermove", (pointer: Phaser.Input.Pointer) => {
    this.pointerTargetX = pointer.x;
});
this.input.on("pointerdown", (pointer: Phaser.Input.Pointer) => {
    this.pointerTargetX = pointer.x;
});
```

```
// CHALLENGE 1: a key press takes over from the pointer, until the pointer
moves again
if (this.cursors.left.isDown) {
    this.pointerTargetX = null;
    this.basket.move(-1);
} else if (this.cursors.right.isDown) {
    this.pointerTargetX = null;
    this.basket.move(1);
} else if (this.pointerTargetX !== null) {
    this.moveTowardsPointer(this.pointerTargetX);
} else {
    this.basket.move(0);
}
```

```
private moveTowardsPointer(targetX: number): void {
  const distance = targetX - this.basket.x;
  if (Math.abs(distance) < CLOSE_ENOUGH) {
    this.basket.move(0);
  } else {
    this.basket.move(Math.sign(distance));
  }
}
```

`Basket` is unchanged - the scene still only calls `move(-1 | 0 | 1)`. That is the pay-off of the chapter's design: a new way to control the basket, and not one line of the basket's class changes.

Look for:

- `pointerdown` as well as `pointermove`: a touch screen has no "move without pressing"
- a dead zone (`CLOSE_ENOUGH`). Without it the basket overshoots by a few pixels every frame and jitters from side to side - a good thing to let students see, then explain
- `pointerTargetX` reset in `init()` - otherwise a restarted game remembers where the pointer was
- a common simpler answer sets `this.basket.x = pointer.x`. It works, but the basket teleports and ignores its speed; the challenge asks it to slide. Some students use `this.physics.moveTo(basket, pointer.x, basket.y, SPEED)` - fine, if they stop it on arrival

Tested with the harness: mouse moved to $x = 100$, the basket stopped at 96; clicked at $x = 650$, it stopped at 656; LEFT held, it moved left and the pointer target was cleared.

2. Combo scoring

Project: `solutions/ch14_challenge_2_combo_scoring` (from `ch14_catcher_simple`)

Goal: a piece of game state with a clear reset rule, and feedback with a tween.

`src/scenes/GameScene.ts`

```
// CHALLENGE 2: each fruit in a row is worth 10 more than the last, up to
MAX_COMBO times 10
this.combo = this.combo + 1;
const points = FRUIT_POINTS * Math.min(this.combo, MAX_COMBO);
this.showPoints(fruit.x, fruit.y, points);
```

The combo breaks in one place - `loseLife()` , which both a dropped fruit and a caught bomb already go through:

```
this.sound.play(HURT_SOUND);
this.combo = 0; // CHALLENGE 2: a dropped fruit or a caught bomb
breaks the combo
this.lives = this.lives - 1;
```

The floating score is a text that tweens up and fades, then destroys itself:

```
this.tweens.add({
  targets: text,
  y: y - 80,
  alpha: 0,
  duration: 700,
  onComplete: () => {
    text.destroy();
  },
});
```

Look for:

- the reset in **one** place, reached by every way of missing - students who reset in both `catchBomb` and the drop code often forget one
- `this.combo = 0` in `init()` as well (the restart trap again)
- `text.destroy()` in `onComplete` : without it, every caught fruit leaves an invisible text behind
- reading `fruit.x` / `fruit.y` **before** `fruit.destroy()` . (A destroyed Phaser object still has its last `x` and `y` , so reading them after works too - but it is a bad habit.)

Tested with the harness: three fruit caught in a row scored $10 + 20 + 30 = 60$, a caught bomb set the combo to 0, and the next fruit scored 10.

3. Zig-zag enemies

Project: `solutions/ch14_challenge_3_zig_zag` (from `ch14_shooter_intermediate`)

Goal: a new wave pattern, combining a velocity with a repeating tween; a difficulty ramp.

The zig-zag is a yoyo tween on `x` that repeats for ever, while the velocity carries the ship down:

`src/objects/Enemy.ts`

```
public zigZag(width: number, time: number): void {
  this.scene.tweens.add({
    targets: this,
    x: { from: this.x - width / 2, to: this.x + width / 2 },
    duration: time,
    ease: "Sine.easeInOut",
    yoyo: true,
    repeat: -1,
  });
}
```

`src/scenes/GameScene.ts`

```
// CHALLENGE 3: three patterns in turn - row, stream, zig-zag, row, ...
const pattern = this.wave % 3;
if (pattern === 1) {
  this.rowWave(size, ROW_SPEED + extraSpeed);
} else if (pattern === 2) {
  this.streamWave(size, STREAM_SPEED + extraSpeed);
} else {
  this.zigZagWave(size, ZIGZAG_SPEED + extraSpeed);
}
```

```
const width = Math.min(ZIGZAG_WIDTH + this.wave * ZIGZAG_WIDTH_PER_WAVE,
  ZIGZAG_MAX_WIDTH);
const time = Math.max(ZIGZAG_TIME - this.wave * ZIGZAG_TIME_PER_WAVE,
  ZIGZAG_MIN_TIME);
```

"Wilder" is wider (up to a limit) and quicker (down to a limit). `Math.min` and `Math.max` are the limits - without them, wave 30 would be unplayable.

The important point to draw out: `Enemy.kill()` already calls `killTweensOf(this)`, so the endless zig-zag tween is stopped when the ship is shot down or leaves - and the pool hands out a clean ship next time. Tested with the harness: a zig-zag wave (wave 3), then the row wave that followed reused the same enemy objects, and every one of them had 0 tweens and flew in a straight row.

Look for:

- ships spawned far enough from the edges for the whole sway to stay on screen
- an answer that sways with `Math.sin(time)` in a `preUpdate()` is also good - check that it stores the ship's starting x and resets it in `spawn()`, because a pooled enemy keeps its fields
- a `repeat: -1` tween **without** `killTweensOf` in `kill()` - the classic pooled-object bug. Ask what the next row wave looks like

4. Shield power-up

Project: `solutions/ch14_challenge_4_shield` (from `ch14_shooter_advanced`)

Goal: extend a union type and follow the compiler to every place that must change; a picture made in code; a new rule for being hit.

Adding `"shield"` to `PowerUpKind` and nothing else gives exactly the two errors the hint promises:

```
TS2741 [ERROR]: Property 'shield' is missing in type '{ spread: string; rapid: string; life: string; }' but required in type 'Record<PowerUpKind, string>'.
```

```
const TEXTURES: Record<PowerUpKind, string> = {  
    ~~~~~
```

```
TS2345 [ERROR]: Argument of type '"spread" | "rapid" | "shield"' is not assignable to parameter of type 'WeaponPowerUp'.
```

```
    Type '"shield"' is not assignable to type 'WeaponPowerUp'.
```

```
    this.ship.powerUp(powerUp.kind, this.time.now);  
    ~~~~~
```

The first needs a picture, which `PreloadScene` draws with `Graphics` (a blue orb for the pickup, and a see-through bubble for the ship). The second needs a new branch in `collect()` :

`src/scenes/GameScene.ts`

```
} else if (powerUp.kind === "shield") {  
    this.ship.giveShield();          // CHALLENGE 4  
} else {
```

The ship owns its bubble, and keeps it in place in a new `preUpdate()` :

`src/objects/PlayerShip.ts`

```
preUpdate(_time: number, _delta: number): void {  
    this.bubble.setPosition(this.x, this.y);  
}
```

and `shipHit()` checks the shield first:

`src/scenes/GameScene.ts`

```
if (this.ship.hasShield()) {  
    this.ship.popShield();  
    this.sound.play(HIT_SOUND);  
    this.ship.makeInvulnerable();  
    return;  
}
```

The shield never wears off, so it is a `boolean` , not an "until" time like spread and rapid. The HUD needed no new code: `describeWeapon()` adds `"SHIELD"` , and the existing `WEAPON_CHANGED` event carries it. The menu legend gained a fourth entry.

Look for:

- the `KINDS` array updated too (the compiler does **not** catch this one - without it, shields never drop). A good question for the class: why can the compiler check the `Record` but not the array?
- the moment of safety after the shield pops - without it, two bullets arriving together take the shield *and* a life
- the bubble drawn above the ship (made after it), and hidden when the shield pops

Tested with the harness: a shield power-up collected (HUD showed SHIELD, bubble visible), an enemy bullet popped it with lives still 3, a second bullet two seconds later cost a life.

5. Smart bomb

Project: `solutions/ch14_challenge_5_smart_bomb` (from `ch14_shooter_advanced`)

Goal: act on every live member of a pool; keep the wave count correct; add a HUD value through the event system.

`src/scenes/GameScene.ts`

```

private useBomb(): void {
    if (this.bombs === 0 || this.phase === "over") {
        return;
    }
    this.bombs = this.bombs - 1;
    this.events.emit(BOMBS_CHANGED, this.bombs);
    this.cameras.main.flash(BOMB_FLASH_TIME);

    const { width, height } = this.scale;
    for (const enemy of this.enemies.getMatching("active", true) as Enemy[]) {
        if (enemy.x >= 0 && enemy.x <= width && enemy.y >= 0 && enemy.y <=
height) {
            this.explode(enemy.x, enemy.y);
            enemy.kill();
            this.addScore(ENEMY_POINTS);
            this.enemyGone();
        }
    }
    for (const bullet of this.enemyBullets.getMatching("active", true) as
EnemyBullet[]) {
        bullet.kill();
    }
    if (this.phase === "boss" && this.boss.active) {
        this.boss.weaken(BOMB_BOSS_DAMAGE);
        this.events.emit(BOSS_HEALTH_CHANGED, this.boss.getHealth(),
this.boss.getMaxHealth());
    }
}

```

The two traps the hint points at:

- **"on screen"** - `getMatching("active", true)` also returns ships that are active but waiting off the edge: a row wave's ships above the top, a swoop's ships queued at the start of the path. They should not explode
- `enemiesLeft` - every destroyed ship goes through `enemyGone()`, exactly as a shot-down ship does, so the count stays right and the next wave starts exactly once. The tempting shortcut, `this.enemiesLeft = 0`, is wrong: in a stream wave the ships still to come would then count it below zero, and the wave would never end

The boss "loses a chunk but survives" through a new `Boss.weaken()`, which never takes the last point of health. The HUD shows the bombs through a new `BOMBS_CHANGED` event, and `HudData` gained a `bombs` field for the starting value.

Look for:

- the new listener removed in the HUD's `SHUTDOWN` handler, like the others
- `bombs` reset in `init()`
- `keyboard.on("keydown-B")` , not `once` - it is used up to three times

Tested with the harness: in a stream wave with 2 of 6 ships on screen, B destroyed those 2 (`enemiesLeft` 6 to 4) and the wave still ended when the other 4 had gone; pressed while a swoop's ships were still off screen, it destroyed none; against a 30-health boss it took off 10 and the boss lived; a fourth press did nothing.

6. Second player

Project: `solutions/ch14_challenge_6_second_player` (from `ch14_shooter_intermediate`)

Goal: turn "the player" into data the scene loops over; share a pool between two owners.

Everything that belongs to one player goes in one object:

`src/scenes/GameScene.ts`

```
interface Player {
  name: string;
  ship: PlayerShip;
  left: Phaser.Input.Keyboard.Key;
  right: Phaser.Input.Keyboard.Key;
  fire: Phaser.Input.Keyboard.Key;
  score: number;
  lives: number;
  hudText: Phaser.GameObjects.Text;
}
```

and the old one-player code runs once per player still in the game:

```
for (const player of this.livingPlayers()) {
  if (player.left.isDown) {
    player.ship.move(-1);
  } else if (player.right.isDown) {
    player.ship.move(1);
  } else {
    player.ship.move(0);
  }
}
```

`PlayerShip` does not change at all - it never knew its keys, which is the point of the hint.

Both ships share the bullet pool (made twice as big). A bullet remembers who fired it:

`src/objects/Bullet.ts`

```
public fire(x: number, y: number, owner: number): void {
  this.owner = owner;
  this.enableBody(true, x, y, true, true);
}
```

`src/scenes/GameScene.ts`

```
const shooter = this.players[bullet.owner]; // CHALLENGE 6: read before
kill(), to be clear
```

The stream wave aims at a random living player

(`Phaser.Utls.Array.GetRandom(living)`). Each ship has its own overlap with the enemies, so the callback knows which player was hit. A player with no lives has their ship switched off with `disableBody(true, true)`; when `livingPlayers()` is empty, the game ends, and `GameOverScene` gets `scores: number[]` and says who won (or that it was a draw).

Look for:

- no duplicated "player 2" copies of methods - a loop, or a method taking a `Player`
- the owner set on **every** fire: a pooled bullet keeps its last owner otherwise, and the wrong player scores
- the stream wave not aiming at a player who is out
- a second `PlayerShip` class for player 2, or `if (player === 2)` inside `PlayerShip`, both miss the point of the hint - worth discussing

Tested with the harness: both ships moved on their own keys; bullets fired as player 2 scored for player 2; player 2 knocked out (ship gone, HUD "OUT") while player 1 played on; the next stream wave dropped all five ships above player 1; when player 1 was out too, the game over screen said "Player 2 wins!".

Slides

Chapter 14

Catch, avoid and shoot

One genre, three games



1 / 20

Today

- a player on one line; spawning on a timer
- **object pools** - and the price of reuse
- waves: tweens, timers, a curved path
- fire rate, invulnerability, explosions, a starfield
- growing a game: levels as data, a boss, power-ups, a HUD scene

2 / 20

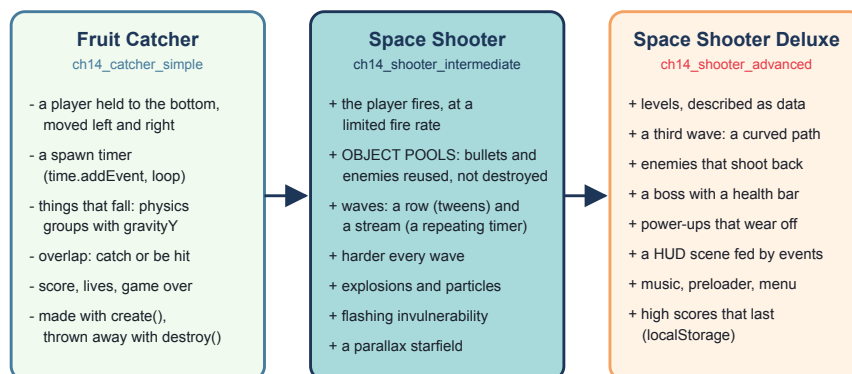
The genre

- the player can do almost nothing - the game is in **what comes down the screen**
- **readable** threats: a bomb looks like a bomb
- **rising pressure**: easy, then harder and harder
- **fairness**: small hit boxes, a moment of safety, no point-blank shots

3 / 20

Three versions

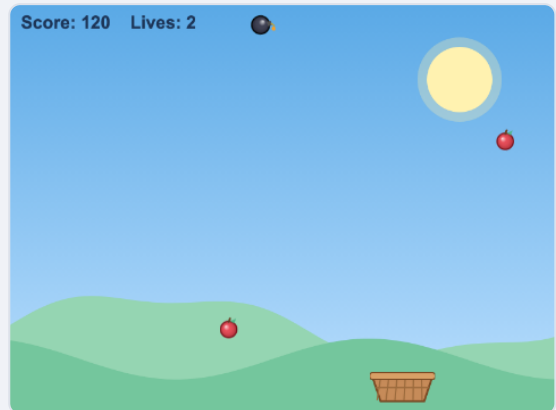
One game, three times - each version keeps the last one's ideas and adds more



4 / 20

Try it: Fruit Catcher

- build and play `ch14_catcher_simple`
- change `SPAWN_DELAY`, `BOMB_CHANCE`, `FALL_GRAVITY` - how does it feel?
- why does a fruit brushing the side not count?



5 / 20

A timer, and things that fall

```
this.fruit = this.physics.add.group({ gravityY: FALL_GRAVITY });

this.spawnTimer = this.time.addEvent({
  delay: SPAWN_DELAY,
  loop: true,
  callback: () => {
    this.dropSomething();
  },
});

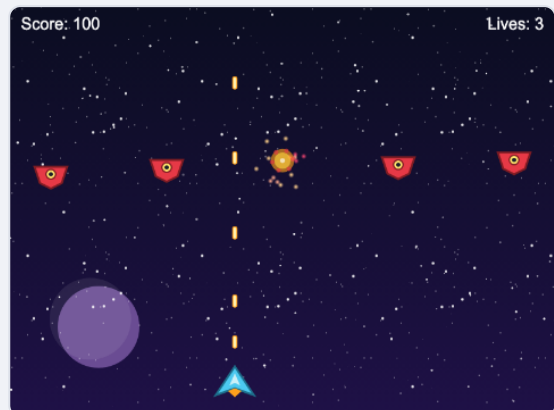
this.physics.add.overlap(this.basket, this.fruit, (_basket, fruit) => {
  this.catchFruit(fruit as Phaser.Physics.Arcade.Sprite);
});
```

`loop: true` - for ever; `repeat: n` - $n + 1$ times; `remove()` - stop

6 / 20

Try it: Space Shooter

- hold SPACE: about 5 shots a second
- how many bullet **objects** does a minute of firing need?
- (we measured: **9**)

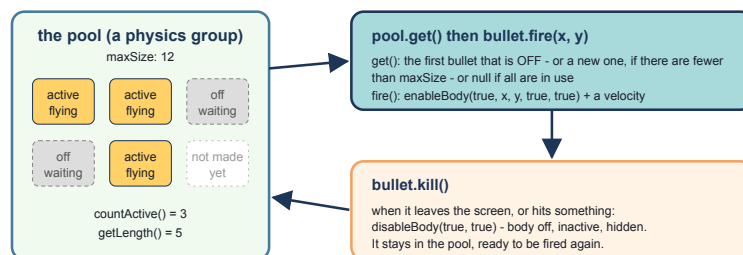


7 / 20

Why pool?

- make + destroy hundreds of objects a minute -> the **garbage collector** -> stutter
- instead: make a few, **switch them off**, switch them on again

An object pool: the same few bullets, switched on and off again and again



No new objects while the game runs, and nothing for the garbage collector to clear up - just switches.

8 / 20

A pool in Phaser

```
this.bullets = this.physics.add.group({ classType: Bullet, maxSize: BULLET_POOL_SIZE });

// firing
const bullet = this.bullets.get() as Bullet | null;
if (bullet === null) {
    return;
}
bullet.fire(this.ship.x, this.ship.y - MUZZLE_OFFSET);

// in Bullet
this.enableBody(true, x, y, true, true); // on
this.disableBody(true, true);           // off
```

9 / 20

Three switches

Switch	Off means
active	no <code>preUpdate()</code> ; free to the pool
visible	not drawn
body.enable	not moved, overlaps nothing

`killAndHide` = the first two only. **Demo:** an invisible bullet that still shoots down ships.

10 / 20

The price of reuse

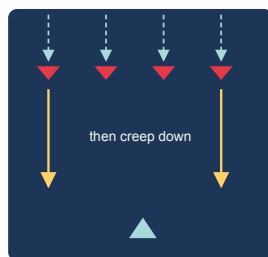
```
public kill(): void {  
    this.scene.tweens.killTweensOf(this);  
    this.disableBody(true, true);  
}
```

- a reused object brings back **everything** you do not reset
- tweens, velocities, angles, tints, fields
- "only goes wrong the second time" = this

11 / 20

Waves

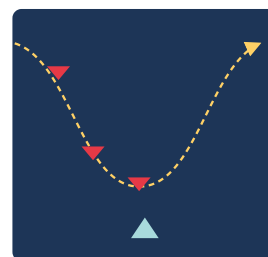
Three wave patterns



row
tweens, each with a longer delay



stream
a timer with repeat: size - 1



swoop (advanced)
a Spline path; a tween moves 0 to 1 along it

- row: tweens with `delay: i * ROW_STAGGER`
- stream: `addEvent({ delay, repeat: size - 1, ... })`
- count the wave down: `enemiesLeft`, one `enemyGone()`

12 / 20

Hit, flash, safe

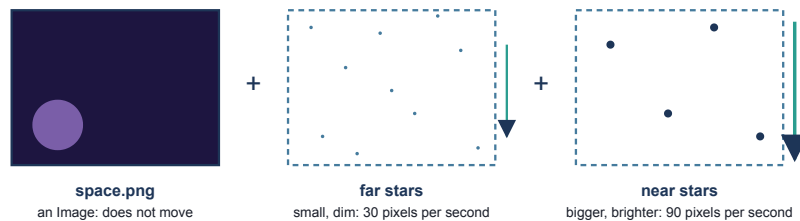
```
this.scene.tweens.add({
  targets: this,
  alpha: 0.15,
  duration: FLASH_TIME,
  yoyo: true,
  repeat: INVULNERABLE_TIME / (FLASH_TIME * 2) - 1,
  onComplete: () => {
    this.setAlpha(1);
    this.invulnerable = false;
  },
});
```

- a process callback: `() => !this.ship.isInvulnerable()`

13 / 20

A parallax starfield

A starfield in three layers - a picture and two TileSprites



Each TileSprite repeats a 256 x 256 star picture; changing `tilePositionY` slides the picture inside it.

`TileSprite` + `tilePositionY`; textures drawn with `Graphics` + `generateTexture`

14 / 20

Try it: Space Shooter Deluxe

- three levels, a boss each
- enemies shoot back; power-ups
- HUD scene, music, high scores
- open `levels.ts` - make level 1 easier



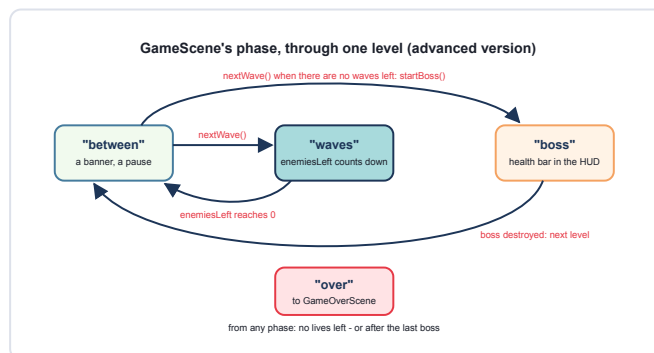
15 / 20

Levels as data, and a phase

```
export type WavePattern = "row" | "stream" | "swoop";

export interface WaveData {
  pattern: WavePattern;
  size: number;           // how many ships
  speed: number;          // pixels per second (swoop: ms)
}

type Phase = "waves" | "between" | "boss" | "over";
```



16 / 20

Along a curve

```
// WaveSpawner.swoop()
const path = new Phaser.Curves.Spline(points);
enemy?.followPath(path, duration, i * SWOOP_GAP);

// Enemy.followPath(): a tween moves pathProgress from 0 to 1
this.scene tweens.add({
  targets: this,
  pathProgress: 1,
  duration: duration,
  delay: delay,
});

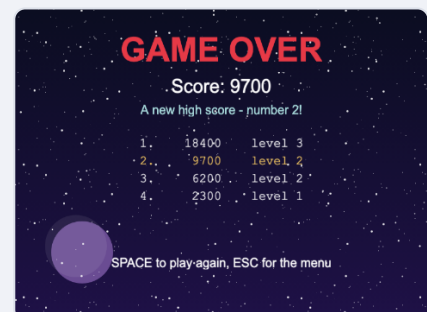
// Enemy.preUpdate(): every frame, go to that point on the curve
this.path.getPointAt(this.pathProgress, this.point);
this.setPosition(this.point.x, this.point.y);
```

A tween can move **any** number.

17 / 20

Boss, power-ups, HUD

- boss: an entry tween, then `yoyo: true, repeat: -1`; a hit flash with `setTint(0xffffffff).setTintMode(Phaser.TintModes.FILL)` (no `setTintFill` in Phaser 4)
- power-ups: "active until" `times`; `Record<PowerUpKind, string>`; narrowing in `collect()`
- HUD: `this.events.emit(SCORE_CHANGED, this.score)` - and `off()` on `SHUTDOWN`



18 / 20

Summary

- `time.addEvent` spawns; physics groups give members the same motion
- pool what you make **many of, often:** `classType` , `maxSize` , `get()` , `enableBody` / `disableBody`
- reset **everything** on reuse; `killAndHide` leaves the body on
- waves: staggered tweens, repeating timers, paths - counted down to zero
- fire rate and power-ups = stored times
- grow with data, phases, small classes, and a HUD that listens

19 / 20

Challenges

1. **Pointer control** - the basket follows the mouse or a finger (catcher)
2. **Combo scoring** - 10, 20, 30... for fruit in a row (catcher)
3. **Zig-zag enemies** - a third wave pattern (intermediate)
4. **Shield power-up** - takes the next hit (advanced)
5. **Smart bomb** - B clears the screen, three a game (advanced)
6. **Second player** - A / D / W, two scores (intermediate)

Next: **Chapter 15 - Platformer**

20 / 20